

detect_bci_artifacts.m

```
function [bad_chan, artifact_bool] = detect_bci_artifacts(BCI)
%{
function detects artifacts in raw EEG data
Bad channels are defined as those with variance > 5z compared to other
channels
artifacts are defined as trials with any values over 100 microvoltz
requirements: Fieldtrip-201904019
input: BCI-BCI data structure
output: bad_chan - vector of noisy channels
        artifact_bool - vector 1 if trial contains artifact 0 if trial does
        not contain artifact
%}

%get percentage of non-artifact trials
%add obj information
bad_chan = [];
artifact_bool = [];
BCI_runs = unique([BCI.TrialData(:).runnumber]);
for run = 1:length(BCI_runs)
    run_ind = find([BCI.TrialData(:).runnumber] == BCI_runs(run));
    trials = BCI.data(run_ind);
    artifact_found = zeros(length(trials),1);

    %look for bad electrodes
    artifact_elec = 1;
    bad_chan_run = [];
    while artifact_elec
        trials_full = cat(2,trials{:});
        trials_full(bad_chan_run,:) = 0;
        trials_full_filt = ft_preproc_bandpassfilter(trials_full,BCI.SRATE,[8 30]);
        elec_var = var(trials_full_filt,[],2);
        elec_z = zscore(elec_var);
        tmp_artifact = find(elec_z > 5);
        if isempty(tmp_artifact)
            artifact_elec = 0;
        else
            bad_chan_run = [bad_chan_run;tmp_artifact];
        end
    end
end

for trial = 1:length(trials)
    trial_filt = ft_preproc_bandpassfilter(trials{trial},BCI.SRATE,[8 30]);
    %constrict to feedback exclusively
```

```

        trial_filt = trial_filt(:,(4*BCI.SRATE + 1):BCI.TrialData(run_ind(trial)).resultind);
        trial_filt(bad_chan_run,:) = 0;
        artifact_found(trial) = logical(sum(abs(trial_filt(:)) > 100));
    end
    artifact_bool = [artifact_bool;artifact_found];
    bad_chan = [bad_chan;bad_chan_run];
end%for BCI run

bad_chan = unique(bad_chan);

end%function

```

make_ERD_figures.m

```

function make_ERD_figures(data_path)
%{
function calculates event related desynchronization (ERD) for the 4 trial
types (left, right, up, down) for each BCI session then averages these
values across subjects and sessions
requires: FieldTrip-20190419
         Rob Campbell (2020). raacampbell/shadedErrorBar
(https://github.com/raacampbell/shadedErrorBar), GitHub. Retrieved November 4, 2020.
         Pekka Kumpulainen (2020). tight_subplot(Nh, Nw, gap, marg_h, marg_w)
(https://www.mathworks.com/matlabcentral/fileexchange/27991-tight\_subplot-nh-nw-gap-marg\_h-marg\_w), MATLAB Central File Exchange. Retrieved November 4, 2020.
input: data_path: where datafiles are stored

output: Panels A and B from figure 3 of data paper
%}

mfile = mfilename('fullpath');
cur_path = mfile(1:(end-16));
path.data = [data_path,'\'];
path.save = [cur_path,'ERDs\'];

if ~exist(path.save,'dir')
    mkdir(path.save)
end

```

```

subfiles = dir(path.data);
subfiles = {subfiles(~[subfiles.isdir]).name};
%data for chan interpolation
layout = load([cur_path,'layoutfull.mat']);
neighbours = layout.neighbours;
BCIbase = load([cur_path,'tfrStructBase.mat']);
elec = BCIbase.BCI.elec;

for sub_num = 1:62
    for sess_num = 1:11
        file_fn = ['S',num2str(sub_num),'_Session_',num2str(sess_num),'.mat'];

        try
            tmp = load([path.data,file_fn]);
        catch
            continue
        end
        fprintf('Processing Subject %i, Session %i...\n',sub_num,sess_num)
        BCI = tmp.BCI;

        %do channel repair
        if ~isempty(BCI.chaninfo.noisechan)
            bci_raw = [];
            bci_raw.label = BCI.chaninfo.label;
            bci_raw.time = BCI.time;
            bci_raw.trial = BCI.data;
            cfg = [];
            cfg.method = 'spline';
            cfg.badchannel = bci_raw.label(BCI.chaninfo.noisechan);
            cfg.neighbours = neighbours;
            cfg.elec = elec;
            interp_data = ft_channelrepair(cfg,bci_raw);
            BCI.data = interp_data.trial;
        end

        %filter data
        trial_lens = cell2mat(cellfun(@(x) length(x), BCI.data,'UniformOutput',0));
        session_dat = cat(2,BCI.data{:});
    end
end

```

```

session_filt = ft_preproc_highpassfilter(session_dat,1000,1);
session_filt = ft_preproc_bandpassfilter(session_filt,1000,[8 14]);
if sum(isnan(session_filt(:)))
    error('error: nans present')
end
session_hilbert = abs(hilbert(session_filt));

%break into individual trials
hilbert_matrix = NaN(length(BCI.data),62,11041);
cur_sum = 0;
for trial = 1:length(BCI.data)
    trial_dat = session_hilbert(:, (cur_sum + 1):(cur_sum + trial_lens(trial)));
    hilbert_matrix(trial,:,1:trial_lens(trial)) = trial_dat;
    hilbert_matrix(trial,:,BCI.TrialData(trial).resultind:end) = NaN;
    cur_sum = cur_sum + trial_lens(trial);
end

%get trial indexes
good_trials = find(~[BCI.TrialData(:).artifact]);

%loop through trial types to get ERD matrix
ERD_matrix = NaN(4,62,11041);
for trial_type = 1:4
    if trial_type < 3
        trial_ind = find((([BCI.TrialData(:).targetnumber] == trial_type) &
([BCI.TrialData(:).tasknumber] == 1) & ([BCI.TrialData(:).result] == 1)) ;
    else
        trial_ind = find((([BCI.TrialData(:).targetnumber] == trial_type) &
([BCI.TrialData(:).tasknumber] == 2) & ([BCI.TrialData(:).result] == 1)) ;
    end
    trial_ind = intersect(trial_ind,good_trials);
    %check for outlier trials
    ERD_var_feed = zscore(nanvar(hilbert_matrix(trial_ind,[26,30],4001:end),[],3));
    ERD_var_feed = max((ERD_var_feed),[],2);
    valid_trials_feed = find(ERD_var_feed < 3.5);
    ERD_var_base =
zscore(nanvar(hilbert_matrix(trial_ind,[26,30],1000:2000),[],3));
    ERD_var_base = max((ERD_var_base),[],2);
    valid_trials_base = find(ERD_var_base < 3.5);
    valid_trials = intersect(valid_trials_feed,valid_trials_base);

```

```

    ERD_matrix(trial_type, :, :) = nanmean(hilbert_matrix(trial_ind(valid_trials), :, :), 1);
end

time_ind = find(BCI.time{1} >= -1000 & BCI.time{1} <= 0);
ERD_base = nanmean(ERD_matrix(:, :, time_ind), 3);

ERD_matrix = ((ERD_matrix - ERD_base) ./ ERD_base) .* 100;
ERD_time = ERD_matrix(:, [26, 30], :);

%%
%look at topos
hilbert_base = nanmean(hilbert_matrix(:, :, time_ind), 3);
hilbert_feed = NaN(length(BCI.data), 62);
for trial = 1:length(BCI.data)
    trial_feed = hilbert_matrix(trial, :, 4001:BCI.TrialData(trial).resultind);
    hilbert_feed(trial, :) = nanmean(trial_feed, 3);
end

ERD_topo = NaN(4, 62);
for trial_type = 1:4
    if trial_type < 3
        trial_ind = find((BCI.TrialData(:).targetnumber == trial_type) &
            (BCI.TrialData(:).tasknumber == 1) & (BCI.TrialData(:).result == 1)) ;
    else
        trial_ind = find((BCI.TrialData(:).targetnumber == trial_type) &
            (BCI.TrialData(:).tasknumber == 2) & (BCI.TrialData(:).result == 1)) ;
    end
    trial_ind = intersect(trial_ind, good_trials);

    ERD_topo(trial_type, :) = ((nanmean(hilbert_feed(trial_ind, :), 1) -
        nanmean(hilbert_base(trial_ind, :), 1)) ./ nanmean(hilbert_base(trial_ind, :), 1)) .* 100;
end

save([path.save, file_fn], 'ERD_time', 'ERD_topo')

end%sess
end%sub num

%%

```

```

%now load erds and make figures
clearvars
mfile = mfilename('fullpath');
cur_path = mfile(1:(end-16));
path.save = [cur_path,'ERDs\'];

%size info
num_subs = 62;
ERD_time = 11041;
num_chan = 62;
num_types = 4;
num_sess = 11;
%group arrays
group_ERD_time = NaN(num_subs,num_sess,num_types,2,ERD_time);
group_ERD_topo = NaN(num_subs,num_sess,num_types,num_chan);
%subject info for future indexing
sub_sess = zeros(num_subs,1);

for sub_num = 1:62
    for sess_num = 1:11

        file_fn = ['S',num2str(sub_num),'_Session_',num2str(sess_num),'.mat'];

        try
            tmp = load([path.save,file_fn]);
        catch
            continue
        end
        fprintf('Processing Subject %i, Session %i...\n',sub_num,sess_num)

        %add session number to indexing
        sub_sess(sub_num) = sub_sess(sub_num) + 1;

        %add data to arrays
        group_ERD_time(sub_num,sess_num, :, :) = tmp.ERD_time;
        group_ERD_topo(sub_num,sess_num, :, :) = tmp.ERD_topo;

    end%sess_num

```

```
end%sub_num
```

```
%%
```

```
%average over all sessions, then average subjects, plot ERD
```

```
final_ERD_time = squeeze(nanmean(group_ERD_time,2));
```

```
k = 100;%smoothing parameter for moving average
```

```
time = -2:(1/1000):9.04;
```

```
time_ind = 1001:8041;
```

```
trial_types = {'Right','Left','Up','Down'};
```

```
PDMS = {'LR','UD'};
```

```
color_cell = {[.75,.12,0.12],[0,0.45,0.74]};
```

```
figure('position',[190 567 1050 520])
```

```
ha = tight_subplot(4,1,[.01 .005],[.1 .05],[.15 .05]);
```

```
for t = 1:4
```

```
    axes(ha(t))
```

```
    shadedErrorBar(time(time_ind),movmean(squeeze(nanmean(final_ERD_time(:,t,1,time_ind),1)),k),movmean(squeeze(nanstd(final_ERD_time(:,t,1,time_ind),1))./sqrt(num_sub_s),k),{'markerfacecolor',color_cell{1},'color',color_cell{1}, 'LineWidth',1.5,'LineStyle','-'}, 1)
    hold on
```

```
    shadedErrorBar(time(time_ind),movmean(squeeze(nanmean(final_ERD_time(:,t,2,time_ind),1)),k),movmean(squeeze(nanstd(final_ERD_time(:,t,2,time_ind),1))./sqrt(num_sub_s),k),{'markerfacecolor',color_cell{2},'color',color_cell{2}, 'LineWidth',1.5,'LineStyle','-'}, 1)
```

```
    if t == 4
```

```
        xlabel('Time (s)')
```

```
        ylabel('Event Related Desynchronization (%)')
```

```
    else
```

```
        set(gca,'xtick',[])
```

```
    end
```

```
    yl = ylim;
```

```
    yrange = yl(2) - yl(1);
```

```
    newyl = [(yl(1) -.1*yrange), (yl(2))];
```

```
    yrange = newyl(2) - newyl(1);
```

```

rectangle('position',[-1,newyl(1),2,.05*yrange],'Curvature',0,'FaceColor','r','EdgeColor','r')

rectangle('position',[0,newyl(1),2,.05*yrange],'Curvature',0,'FaceColor','y','EdgeColor','y')

rectangle('position',[2,newyl(1),6.04,.05*yrange],'Curvature',0,'FaceColor','g','EdgeColor'
,'g')
    xlim([-1,6])
    ylim(newyl)
    h1 =plot(NaN,NaN,'Color',color_cell{1},'LineWidth',5);
    h2 =plot(NaN,NaN,'Color',color_cell{2},'LineWidth',5);
    legend([h1 h2],{'C3','C4'});
    title(trial_types{t})
    %set(gca,'FontSize',20,'FontName','Myriad Pro')
    box off
end

```

```

%%
%topo as t-test
layout_tmp = load([cur_path,'layoutfull.mat']);
layout = layout_tmp.layout;
neighbours = layout_tmp.neighbours;
BCIbase = load([cur_path,'tfrStructBase.mat']);
BCI = BCIbase.BCI;

final_ERD_topo = squeeze(nanmean(group_ERD_topo,2));
figure('position',[680 58 560 1040])
ha = tight_subplot(4,1,[.01 .005],[.01 .01],[.01 .01]);

BCI.freq = 1;
BCI.time = 1;

BCI.dimord = 'chan_freq';

for t = 1:4
    axes(ha(t))

    [h,p,ci,stats] = ttest(squeeze(final_ERD_topo(:,t,:)));

```

```

BCI.powspectrm = stats.tstat';

cfg = [];
cfg.layout = layout;
cfg.marker = 'off';
cfg.style = 'fill';
cfg.comment = 'no';
cfg.gridscale = 200;
cfg.numcontour = 15;

ft_topoplotER(cfg,BCI)
c = colorbar;
c.Label.String = 'ERD t-value';
c.Label.Rotation = -90;
c.Label.Position(1) = 4.41;
%c.FontName = 'Myriad Pro';
c.FontSize = 20;
c.LineWidth = 2;
if t == 4
    c.TickLabels = {'0','+1','+2','+3','+4','+5'};
end
ax = gca;

ax.Children(1).LineColor = 'none';%1
for c = 2:5
    if c < 5
        ax.Children(c).LineWidth = 2.5;
    else
        ax.Children(c).LineWidth = 5;
    end
end
end%for t

end%function

```

Run_behavior_analysis.m

```
function run_behavior_analysis(data_path)
%{
function collects behavioral data from the BCI files
input: data_path: where datafiles are stored

output: figures 1 and 2 from data paper
%}
%script setup

path.data = data_path;
subfiles = dir(path.data);
subfiles = {subfiles(~[subfiles.isdir]).name};

%metadata hold information regarding dataset
metadata.subjects = 62;
metadata.sessions = length(subfiles);
md_fields = {'runs','trials','total_time','feedback_time'};%time in seconds

for mdf = 1:length(md_fields)
    metadata.(md_fields{mdf}) = 0;
end

%data matrices to hold collected information
%bci_pvc: subs, sessions, pdm
bci_pvc = NaN(metadata.subjects,11,3);

%percent of artifact trials
artifact_percentage = zeros(1,length(subfiles));

%Number of artifact channels
artifact_chan = zeros(1,length(subfiles));

%proficiency analysis
prof_thresh = [70.0,70.0,40.0];
%found_prof: subs by pdm
```

```

found_prof = zeros(metadata.subjects,3);
%sub_prof: run where proficiency occurs-100 if never happens >66
sub_prof = 100.*ones(metadata.subjects,3);

%%

%loop through subjects
for sub_num = 1:metadata.subjects
    for sess_num = 1:11
        try
            file_fn = ['S',num2str(sub_num),'_Session_',num2str(sess_num),'.mat'];
            fprintf('loading file %s...\n',file_fn)
            tmp = load([path.data,file_fn]);
            BCI = tmp.BCI;

            file = file + 1;

            %get position information
            trialdata = BCI.TrialData;
            %sub_num = trialdata(1).subjectnumber;
            %sess_num = trialdata(1).sessionnumber;

            %add easy info to meta data
            runs = unique([trialdata.runnumber]);
            metadata.runs = metadata.runs + length(runs);
            metadata.trials = metadata.trials + length(trialdata);
            metadata.feedback_time = metadata.feedback_time +
sum([trialdata.triallength]);
            sess_time = cellfun(@(x) size(x,2), BCI.time);
            metadata.total_time = metadata.total_time + sum(sess_time./1000);

            %add session artifact percentage
            artifact_percentage(file) = nanmean([trialdata.artifact]);

            %add number of artifact channels
            artifact_chan(file) =length(BCI.chaninfo.noisechan);

            %get pvc by run for subject/sess/pdm
            run_pvc = NaN(6,3);

```

```

bci_table = struct2table(trialdata);
for pdm = 1:3
    pdm_ind = find(bci_table.tasknumber == pdm);
    pdm_runs = unique(bci_table{pdm_ind,'runnumber'});
    for run = 1:length(pdm_runs)
        run_ind = find(bci_table.runnumber == pdm_runs(run));
        run_table = bci_table(run_ind,:);
        run_pvc(run,pdm) = nanmean(run_table.result).*100;
    end
end

%add session to pvc mat
avg_pvc = nanmean(run_pvc,1);
bci_pvc(sub_num,sess_num,:) = avg_pvc;

%check for proficiency
for pdm = 1:3
    if ~found_prof(sub_num,pdm)
        %3 checks, first block second block avg
        if nanmean(run_pvc(1:3,pdm)) > prof_thresh(pdm)
            found_prof(sub_num,pdm) = 1;
            sub_prof(sub_num,pdm) = (sess_num - 1)*6 + 3;
        elseif nanmean(run_pvc(4:6,pdm)) > prof_thresh(pdm)
            found_prof(sub_num,pdm) = 1;
            sub_prof(sub_num,pdm) = (sess_num - 1)*6 + 6;
        elseif nanmean(run_pvc(:,pdm)) > prof_thresh(pdm)
            found_prof(sub_num,pdm) = 1;
            sub_prof(sub_num,pdm) = (sess_num - 1)*6 + 6;
        end
    end

    end%if proficiency not found
end
catch
    warning('no file %s',file_fn)
end%try to open
end%sess num
end%sub num

%percent artifacts

```

```

figure
subplot(1,2,1)
hist(artifact_chan)
xlim([0,8])
xlabel('Number of noisy channels')
ylabel('Number of sessions')
subplot(1,2,2)
hist(artifact_percentage.*100,20)
xlim([0,8])
ylabel('Number of sessions')
xlabel('Percent of trials with artifacts(%)')
%Data analysis

%bci performance: classification accuracy
color_cell{1} = [0,0.45,0.74];
figure('position',[190 567 1050 520])
pdms = {'LR','UD','2D'};
for pdm = 1:3
    subplot(1,3,pdm)
    hold on

    plot(1:11,nanmean(bci_pvc(:,:,pdm),1),'Marker','.', 'MarkerSize',20,'MarkerFaceColor',color_cell{1}, 'LineStyle','none')
    hold on

    shadedErrorBar(1:11,nanmean(bci_pvc(:,:,pdm),1),nanstd(bci_pvc(:,:,pdm),[],1)./sqrt(size(bci_pvc,1)),{'markerfacecolor',color_cell{1},'color',color_cell{1}, 'LineWidth',1.5,'LineStyle','-'}, 1)
    xlim([1,11])
    xlabel('Session')
    ylabel('Percent valid correct (%)')
    %set(gca,'FontSize',20,'FontName','Myriad Pro')
    box off
    title(pdms{pdm})
end

%bci performance: proficiency plot
figure('position',[190 567 1050 520])
for pdm = 1:3

```

```
[f,x] = ecdf(sub_prof(:,pdm)); %sub_prof = run when reach proficiency,f = % of
subplot(1,3,pdm)
stairs(x./6,f.*100)
xlim([0,11.5])
ylim([0,100])
xlabel('Session')
ylabel('Percent of proficient participants (%)')
end
%population, x = run when it happens

end%function
```